

嵌入式计算机系统与实验 | 作业 7

Shuiyuan@Noroshi

1 中断与低功耗模式

1.1 简单中断实验

MSP430 的中断与 51 单片机的中断整体来说比较相似, 都是中断触发 -> 转至中断向量表 -> 执行中断服务程序 -> 返回原程序. 不过由于 MSP430 的整体结构与外设都比 51 单片机复杂很多, 因此其中断的类型会更多一点, 中断优先级也会更重要一点.

不过从简单的中断开始, 下面部分代码就是实现了利用中断查询按钮的按键状态, 而非使用轮询, 搭配其启用的低功耗模式, 可以在不阻塞主程序的同时更节能, 看看代码:

```
void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;
    P4DIR |= BIT7;
    P4DIR &= ~BIT0;
    P4REN |= BIT0;
    P4OUT |= BIT0;
    P4IES |= BIT0;
    P4IFG &= ~BIT0;
    P4IE |= BIT0;

    __bis_SR_register(LPM4_bits + GIE);
    __no_operation();
}

#pragma vector=PORT4_VECTOR
__interrupt void Port_4(void)
{
    P4OUT ^= BIT7;
    P4IFG &= ~BIT0;
    __delay_cycles(1000000); // assuming the clock is running at 1MHz for a 1 second delay
    P4OUT ^= BIT7;
}
```

第一部分我们把 P4.7(对应 led 灯)设为输出, P4.0(对应按钮)设为输入, 且设置为了上拉电阻. 而 `P4IES |= BIT0` 表示启用了下降沿触发中断(高变低, 即位于下降沿时触发中断), `P4IFG |= ~BIT0` 清除了中断标志位, 而 `P4IE |= BIT0` 表示打开了 P4.0 的中断使能

随后设置 SR 为 4 级低功耗模式, 并且大概了全局中断使能(GIE)位

`#pragma vector=PORT4_VECTOR` 这句话是中断服务程序特有的, 以用来向编译器说明这是中断向量 `__interrupt void Port_4(void)` 也是同理. 中断服务程序在这个例子中尚比较简单, 就是在触发中断之后, 反转 LED 灯电平, 重置中断位, 然后等待 1s 再次反转, 体现就是按钮按一次灯亮一秒.

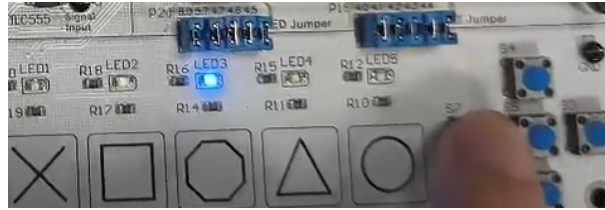


Figure 1: 按下按钮后 LED 点亮 1s, 熄灭, 详细现象见视频 lab7-1

1.2 低功耗实验

低功耗是 MSP430 系列的核心竞争力, 其具体的实现方式就是采用了分级的功耗模式, 其在不同的任务下可以关闭部分外设, 或者通过改变使用的时钟信号以一个更低频的速度运行. 在具体配置方面, 我们可以通过配置 SR 寄存器中的相关位来得到结果, 如下表

SCG1	SCG0	OSCOFF	CPUOFF	功率模式	CPU/MCLK	ACLK	SMCLK
0	0	0	0	正常状态	1	1	1
0	0	0	1	LPM0	0	1	1/0
0	1	0	1	LPM1	0	1	1/0
1	0	0	1	LPM2	0	1	0
1	1	0	1	LPM3	0	1	0
1	1	1	1	LPM4	0	0	0

从这里我们可以看到, 不同功率模式有个很显著的特征是时钟的启用状态不同. 因此, 我们可以通过检测时钟信号的状态来判断低功耗模式是否被正常配置

```
#include <msp430.h>
#include <msp430f6638.h>
int main(void) {
    WDTCTL = WDTPW | WDTHOLD;

    P1DIR |= BIT0;
    P1SEL |= BIT0;

    __bis_SR_register(LPM3_bits);
    // __bis_SR_register(LPM4_bits);
    while (1);
    return 0;
}
```

这个的代码仍然很简单, 就是我们通过配置 SR 的值来切换不同的功耗模式($LPM3_bits = SCG1 + SCG0 + CPUOFF$, $LPM4_bits = SCG1 + SCG0 + CPUOFF + OSCOFF$).

而我们使用的 6638 的实验箱的硬件设计中, 提供了时钟信号作为输出的输出端口: P1.0 在功能选择寄存器设置为 1 时, 可以输出时钟信号, 我们可以用示波器读取其波形. P1.0 位于实验箱上一个叫 BoosterPack 的功能拓展硬件区域.

那我们调试观察一下实验效果, 当我们启用 LPM3 功耗模式时(即上方程序, LPM4 控制的语句被注释), 可以在示波器上读取到一个方波波形, 说明此时的时钟在活跃输出时钟信号. 而在启用 LPM4 功耗模式(即注释掉 LPM3, 不注释 LPM4)时, 我们看不到类似的周期性方波信号.

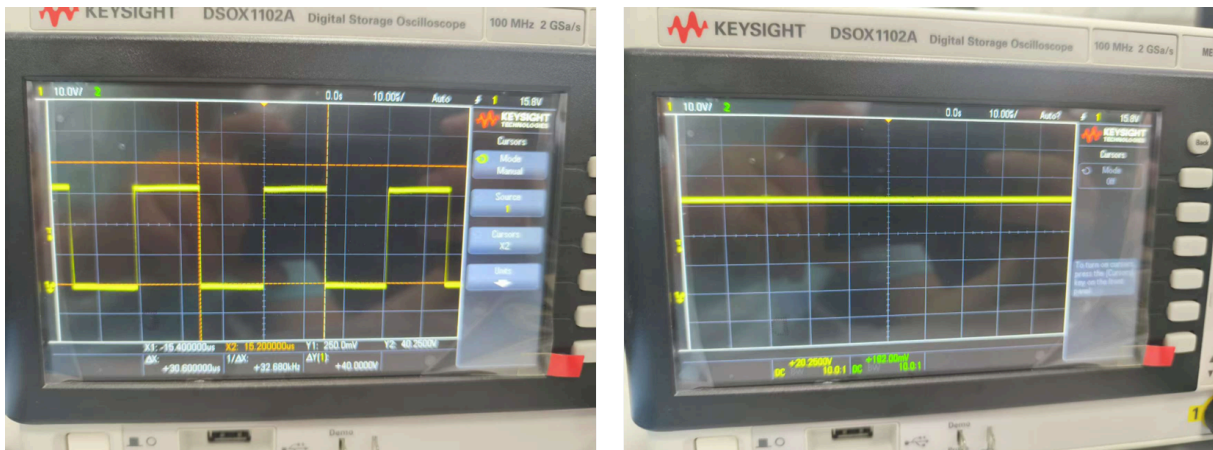


Figure 2: 左图为启用 LPM3 的情况, 并且其工作频率大约在 32~33kHz 的情况下, 这也和实际的时钟频率是类似的
而右图中我们无法从 P1.0 中读取到时钟信号, 也就是此时计时器被关闭了, 这也和我们 LPM4 的实际运行状态是符合的。

2 定时器基本原理

2.1 定时器基础

以 MSP430 的定时器 A 为例, 其是一个 16 为寄存器, 拥有四种操作模式. 同时, 用户还可以为其设置不同的时钟源和分频. 在计时器归零/达到上限时, 会触发计时器中断.

计时器所采用的具体工作模式由其控制寄存器上的 MC 两位决定

MC	效果
00	计时器不工作
01	其从 0 开始, 向上数到 TAxCCR0 的值时触发中断
10	其从 0 开始, 向上数到 0xFFFF 时中断
11	其从 0 开始, 向上数到 TAxCCR0 的值时触发一类中断, 然后从 TAxCCR0 的值又倒数回 0 时触发另一类中断

我们来看一个利用计时器原理的一个实际例子

```
#include <msp430f6638.h>

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;
    P1DIR |= BIT5;
    P4DIR |= BIT5;
    TA0CTL |= MC_1 + TASSEL_2 + TACLK;
    TA0CTL |= ID_3;
    TA0CCTL0 = CCIE;
    TA0CCR0 = 50000;
    __bis_SR_register(LPM0_bits + GIE);
}

#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A (void)
{
    P1OUT ^= BIT5;
}
```

```

P4OUT ^= BIT5;
}

```

我们先看初始配置部分, $TA0CTL |= MC_1 + TASSEL_2 + TACLRL$ 的作用即是设置为正数模式(MC=01), 然后将时钟源设置为子系统时钟(1mhz), 而 TACLRL 则是清除原有的可能对定时器的任何设置.

$TA0CCTL0 = CCIE$ 的作用是打开定时器溢出中断, $TA0CCR0 = 50000$ 代表目前设置的是 50ms 的上限, 但这里为了更好的调整时间间隔, 我们可以使用分频的方法, 例如在这里我们设置了 $TA0CTL |= ID_3$ 代表的是 8 分频, 也就是实际的触发时间是 $50ms * 8 = 0.4s$, 我们也可以调整为例如 ID_4 即是 4 分频, 也就是 0.2s, 以此类推.

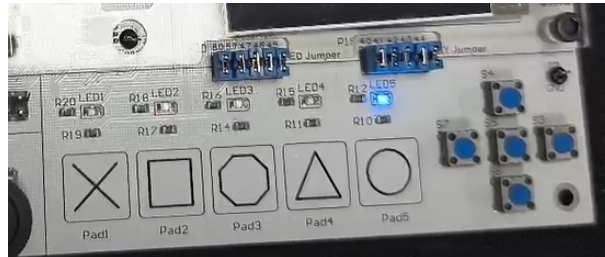


Figure 3: 这个就更看不出来运行效果了, 现象见视频 lab7-3-1, lab7-3-2, lab7-3-1 我们使用的是 4 分频, 所以是 0.2s 亮灯(即 0.4s 一个亮灭周期), 而 lab7-3-2 使用的是 8 分频, 所以是 0.4s 亮灯(0.8s 一个亮灭周期)

2.2 看门狗一

看门狗是一类特殊的计时器, 其效果是其在在上电之后就会自动开始计数, 如果溢出之后就会直接重置. 因此看门狗需要主程序定时向其发送一个信号, 否则就会发送重置信号. 这样就可以在程序跑飞的时候不会卡死.

寄存器有时钟失效保护的, 其可以保证看门狗模式在即是部分时钟出故障的情况下, 也不会影响 WDT 的正常计数, 即使在 SMCLK, ACLK 失效的情况下, 系统也可以自动切换 VLOCLK 给看门狗.

同时, 看门狗为了最大程度确保主程序只在正常运行时才会向其发送信号, 这个看门狗复位的信号为 16 位, 其高位必须是 5A, 否则系统会自动复位.

在之前的实验中我们都有这样一个指令 $WDTCTL = WDTPW + WDT_HOLD$; $WDTPW$ 即是 0x5a00, WDT_HOLD 即是关闭看门狗.

当然, 看门狗也可以是做一个正常的计时器来使用, 比如:

```

#include<msp430f6638.h>

void main(void)
{
    WDTCTL = WDT_ADLY_1000;
    SFRID1 |= WDTIE;
    P4DIR |= BIT5;
    __bis_SR_register(LPM0_bits + GIE);
}

#pragma vector=WDT_VECTOR
__interrupt void WDT_ISR(void)
{
    P4OUT ^= BIT5;
}

```

这里第一行 $WDTCTL = WDT_ADLY_1000$ 其中, $WDT_ADLY_1000 = WDTPW + WDTTMSSEL + WDTCNTCL + WDTIS2 + WDTSSSEL0$, $WDTPW$ 代表的就是高位 5a 的看门狗密码, 而 $WDTTMSSEL0$ 的作用是将看门狗设置为正常

计时器功能, 而 WDTIS2 则代表计时器的溢出值, 而 WDTSEL0 是设定其时钟信号来源. 在这里, 看门狗定时器是 1s 触发一次

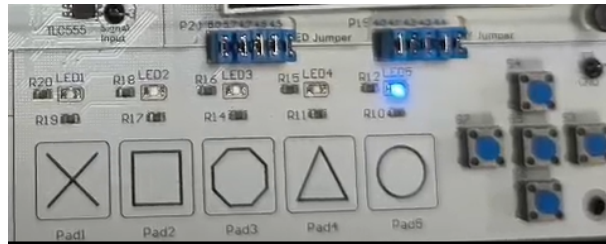


Figure 4: 这里的效果是 1s 为周期的闪光, 具体现象见视频 lab7-4

2.3 看门狗二

接下来我们看看看门狗作为看门狗防跑飞功能的特性

```
#include <msp430f6638.h>

int main(void) {
    WDTCTL = WDTPW | WDTSEL__ACLK | WDTIS_4;
    P4DIR |= BIT5 | BIT6 | BIT7;
    P4OUT |= BIT5 | BIT6 | BIT7;
    __delay_cycles(500000);
    P4OUT &= ~BIT5;
    P4OUT &= ~BIT6;
    P4OUT &= ~BIT7;
    while(1) {
        P4OUT ^= BIT5;
        __delay_cycles(50000);
        WDTCTL = WDTPW | WDTCTL | WDTSEL__ACLK | WDTIS_4;
    }
}
```

这里我们其实主要讲解一下第一行指令和最后一行指令就可以, 第一行 `WDTCTL = WDTPW | WDTSEL__ACLK | WDTIS_4` 的效果是时钟源使用 `ACLK`, 然后 `WDTIS_4` 的时间间隔选择是 1s 溢出.

最后一行则是喂狗指令, 与第一行的初始化指令相似, 只是加上了一个 `WDTCTL`, 其效果就是清除计数器. 如果注释掉就是非喂狗状态, 理论上会反复进入主程序.

这个实验我们的现象是在主程序初始化时, 三个 LED 均会亮起, 但进入死循环后只有一盏灯会反复闪, 借此我们可以判断看门狗是否因为没有喂狗而发起复位信号, 而反复进入主程序.

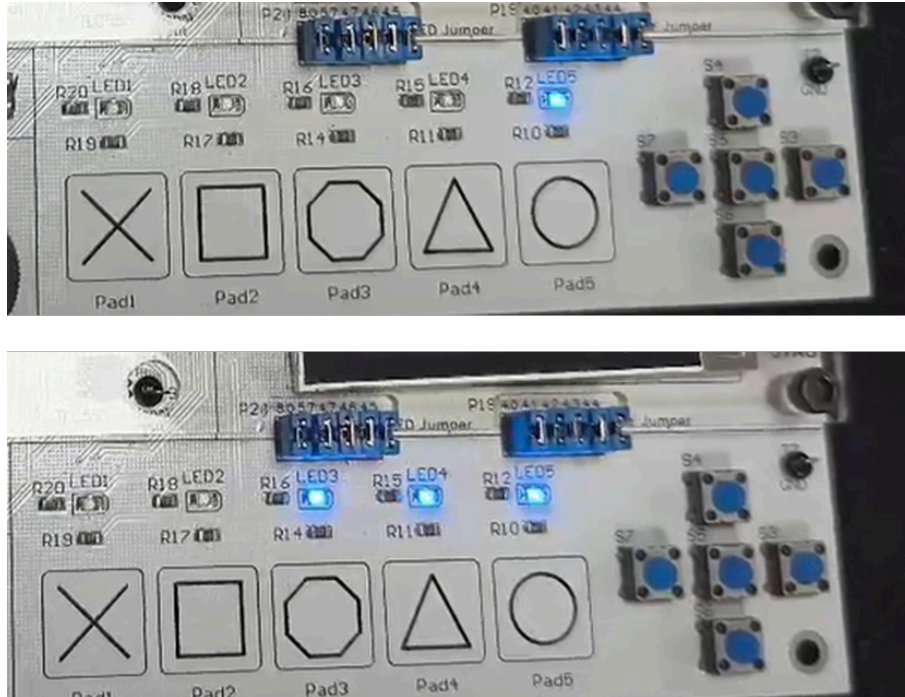


Figure 5: 上图为处在死循环中的状态, 下图为初始化时的状态
具体现象见视频(lab7-5-1 是喂狗, lab7-5-2 是非喂狗)

2.4 定时器二

这个的内容要稍微复杂一点, 而且会涉及多种类型的中断

```
#include <msp430.h>
#include <msp430f6638.h>

volatile unsigned int led_active_count = 0;
volatile unsigned char led_toggle_state = 0;

#define LED1_PORT P4OUT
#define LED1_PIN BIT5

#define LED2_PORT P4OUT
#define LED2_PIN BIT6

#define LED3_PORT P4OUT
#define LED3_PIN BIT7

#define LED4_PORT P5OUT
#define LED4_PIN BIT7

#define LED5_PORT P8OUT
#define LED5_PIN BIT0

#define BUTTON_S3_PORT_IN P4IN
#define BUTTON_S3_PORT_IFG P4IFG
#define BUTTON_S3_PIN BIT4

#define BUTTON_S4_PORT_IN P4IN
#define BUTTON_S4_PORT_IFG P4IFG
#define BUTTON_S4_PIN BIT3
```

```

int main(void) {
    WDTCTL = WDTPW | WDTHOLD;

    P4DIR |= LED1_PIN | LED2_PIN | LED3_PIN;
    P5DIR |= LED4_PIN;
    P8DIR |= LED5_PIN;

    LED1_PORT &= ~LED1_PIN;
    LED2_PORT &= ~LED2_PIN;
    LED3_PORT &= ~LED3_PIN;
    LED4_PORT &= ~LED4_PIN;
    LED5_PORT &= ~LED5_PIN;

    P4DIR &= ~(BUTTON_S3_PIN | BUTTON_S4_PIN);
    P4REN |= (BUTTON_S3_PIN | BUTTON_S4_PIN);
    P4OUT |= (BUTTON_S3_PIN | BUTTON_S4_PIN);
    P4IES |= (BUTTON_S3_PIN | BUTTON_S4_PIN);
    P4IFG &= ~(BUTTON_S3_PIN | BUTTON_S4_PIN);
    P4IE |= (BUTTON_S3_PIN | BUTTON_S4_PIN);

    TA0CTL = TASSEL__ACLK | MC__UP | TACL_R;
    TA0CCR0 = 32767;
    TA0CCTL0 = CCIE;

    __bis_SR_register(LPM0_bits | GIE);

    return 0;
}

#pragma vector=TIMER0_A0_VECTOR
__interrupt void TIMER0_A0_ISR (void) {
    if (led_toggle_state == 0) {
        if (led_active_count >= 1) LED1_PORT |= LED1_PIN;
        if (led_active_count >= 2) LED2_PORT |= LED2_PIN;
        if (led_active_count >= 3) LED3_PORT |= LED3_PIN;
        if (led_active_count >= 4) LED4_PORT |= LED4_PIN;
        if (led_active_count >= 5) LED5_PORT |= LED5_PIN;
        led_toggle_state = 1;
    } else {
        LED1_PORT &= ~LED1_PIN;
        LED2_PORT &= ~LED2_PIN;
        LED3_PORT &= ~LED3_PIN;
        LED4_PORT &= ~LED4_PIN;
        LED5_PORT &= ~LED5_PIN;
        led_toggle_state = 0;
    }
}

#pragma vector=PORT4_VECTOR
__interrupt void Port_4_ISR (void) {
    if (BUTTON_S3_PORT_IFG & BUTTON_S3_PIN) {
        __delay_cycles(20000);
        if (!(BUTTON_S3_PORT_IN & BUTTON_S3_PIN)) {
            if (led_active_count < 5) {
                led_active_count++;
            }
        }
        BUTTON_S3_PORT_IFG &= ~BUTTON_S3_PIN;
    }
}

```

```
if (BUTTON_S4_PORT_IFG & BUTTON_S4_PIN) {  
    __delay_cycles(20000);  
    if (!(BUTTON_S4_PORT_IN & BUTTON_S4_PIN)) {  
        led_active_count = 0;  
        LED1_PORT &= ~LED1_PIN;  
        LED2_PORT &= ~LED2_PIN;  
        LED3_PORT &= ~LED3_PIN;  
        LED4_PORT &= ~LED4_PIN;  
        LED5_PORT &= ~LED5_PIN;  
        led_toggle_state = 0;  
    }  
    BUTTON_S4_PORT_IFG &= ~BUTTON_S4_PIN;  
}  
}
```

前面一大段是提高可读性的宏定义, 初始化部分也就是将 led 灯对应的输出端口设置为输出模式, 将案件对应的端口设置为输入模式, 同时启动中断使能。

后面就是两段中断服务程序, 第一段为计时器的中断服务程序, 其作用就是控制灯的亮灭, 以 2s(1s 亮 1s 灭)为一个周期反复执行。

而第二部分为 P4 口的中断服务程序, 也就是案件的中断程序。由于终端的触发是端口级的, 因此我们需要在进入中断之后还要判断是哪一个按键触发的中断, 如果是 s3 就增加一个灯的点亮, 而 s4 就是重置为 0 盏点亮。

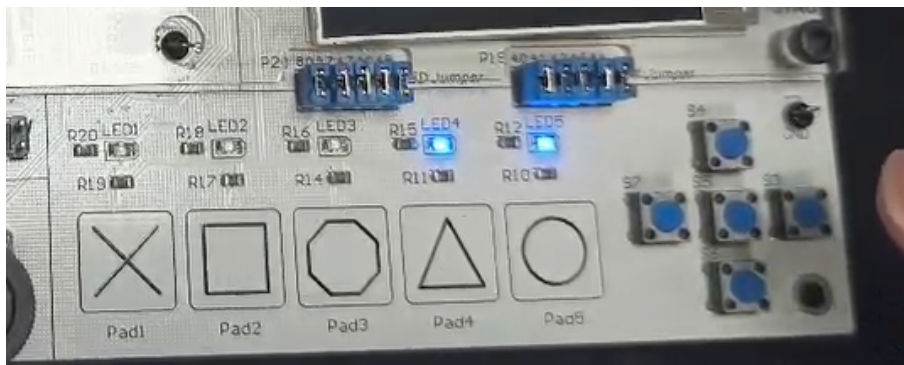


Figure 6: 具体现象见视频 lab7-6